

Introduction to JavaScript

Brief History

JavaScript was created in 1995 by Brendan Eich while working at Netscape. It was originally called Mocha, then LiveScript, and finally JavaScript. The name JavaScript was chosen to capitalize on the popularity of Java at the time, even though **the two languages are not related**.

It was created to make web pages more interactive. It is an interpreted language, meaning it is executed line by line. And it is a client-side language, meaning it runs on the user's computer, not the server.

How to run it

There are a few ways to run JavaScript:

- Using the browser's inspector
- Using the script tag
- Using an external file

Try running the following code in the browser's inspector, line by line:

```
console.log('Hello, World!')
```

```
alert('Hello, World!')
```

Variables & Types

JavaScript is a loosely typed language, meaning you don't have to declare the type of a variable when you create it. You can also change the type of a variable after it has been created.

Historically, JavaScript used the `var` keyword to declare variables. While it still works, it's generally discouraged in modern code. You should use `const` and `let` instead.

```
var x = 5  
let y = 'Hello, World!'
```

```
const z = [1, 2, 3]
const a = { name: 'Foo', age: 30 }
```

`let` is similar to `var`, but it is block-scoped. `const` is similar to `let`, but the value cannot be changed after it is set.

I recommend using `const` whenever possible, and only using `let` when you need to change the value of a variable.

Operators

JavaScript has several operators for performing arithmetic, comparison, and logical operations. These are similar to other programming languages (e.g., `+` for addition, `-` for subtraction, `*` for multiplication, etc.).

One quirk of JavaScript is the `==` operator, which performs type coercion before comparing two values. This can lead to unexpected results, so it is recommended to always use the `===` operator, which compares both the value and the type of the two values.

Falsy values in JavaScript are: `false`, `0`, `NaN`, `""` (empty string), `null`, and `undefined`. Everything else is truthy.

Control Flow

JavaScript has several ways to control the flow of your code, including `if` statements, `switch`, `while`, and `for` loops. These are also similar to most other programming languages.

There are several ways to loop over arrays in JavaScript, including the traditional `for` loop, the `for...in` loop, and the `for...of` loop.

I recommend using the `for...of` loop whenever possible, as it is the most concise and readable way to loop over an array and also `NodeList`, which is a collection of DOM elements (more on that down below).

```
const numbers = [1, 2, 3, 4, 5]

// Traditional for loop
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i])
}

// For...in loop
for (let i in numbers) {
```

```
    console.log(numbers[i])
  }

  // For...of loop
  for (let number of numbers) {
    console.log(number)
  }
```

Functions

Functions are blocks of code that can be reused throughout your program. They can take arguments and return values. They are also first-class citizens in JavaScript, meaning they can be assigned to variables, passed as arguments to other functions, and returned from other functions.

There are several ways to define functions in JavaScript, including function declarations, function expressions, and arrow functions. For beginners, I recommend using arrow functions whenever possible, as they are the most concise and readable way to define functions.

```
// Function declaration
function add(a, b) {
  return a + b
}

// Function expression
const subtract = function(a, b) {
  return a - b
}

// Arrow function
const divide = (a, b) => {
  return a / b
}

// Concise arrow function (one liner with implicit return)
const multiply = (a, b) => a * b
```

Functions are first-class citizens in JavaScript. They can be assigned to variables, passed as arguments to other functions, and returned from other functions. This property enables functional programming patterns, which are very powerful and can be used to write more concise and readable code. We'll see more of this later.

Objects

Objects are collections of related data and functionality, stored as *key-value pairs*. Think of them like dictionaries in Python or maps in other languages. Keys (also called *properties*) are usually strings, and values can be anything: numbers, strings, arrays, functions, or even other objects.

```
const person = {
  name: 'Ada',
  age: 36,
  hobbies: ['math', 'poetry'],
  greet: function() {
    console.log('Hello, my name is ' + this.name)
  }
}

console.log(person.name)    // "Ada"
console.log(person['age'])  // 36
person.greet()              // "Hello, my name is Ada"
```

You can access properties with dot notation (`person.name`) or bracket notation (`person['age']`). Bracket notation is useful when the key is dynamic or not a valid identifier.

A lot of built-in JavaScript functions are actually methods of objects, like

`document.querySelector()` and `console.log()`.

Objects are the foundation of JavaScript. Almost everything in JS is an object (including arrays and functions). They allow us to model real-world entities and are essential for working with the DOM, APIs, and 3rd party libraries.

DOM Manipulation

Accessing the DOM

The DOM, or Document Object Model, is a representation of the HTML document that can be manipulated using JavaScript. We typically access elements in the DOM using

`document.querySelector()` or `document.querySelectorAll()`, but there are other ways to access elements as well:

- `document.querySelector()` (preferred)
- `document.querySelectorAll()` (preferred)
- `document.getElementById()`

- `document.getElementsByClassName()`
- `document.getElementsByTagName()`
- `document.getElementsByName()`
- Open the inspector and click on an element, it will be available in the console as `$0`

Note that `document.querySelectorAll()` returns a *NodeList* (array-like and typically static), while APIs like `getElementsByClassName()` return an *HTMLCollection* (array-like and live). You can loop over a *NodeList* with `for...of` or convert it to an array with `Array.from()`.

Manipulating the DOM

Once we have accessed an element in the DOM, we can manipulate it in several ways:

- Changing attributes: `element.src = 'new-image.jpg'`
- Changing styles: `element.style.backgroundColor = 'red'`
- Changing classes: `element.classList.add('highlight')`
- Changing HTML content: `element.innerHTML = 'new content'` (avoid unless necessary; unsanitized HTML can cause XSS)
- Changing text content: `element.textContent = 'new content'`
- Creating new elements: `document.createElement('div')`
- Appending elements: `parentElement.appendChild(newElement)`
- Removing elements: `element.remove()`

There are **many** other ways to manipulate the DOM, but these are the most common.

Some tips: Prefer toggling CSS classes with `classList` over changing inline styles with `style`. Avoid using `innerHTML` unless necessary, as unsanitized HTML can cause a security issue known as XSS.

Events

Events are actions that occur on a webpage, like clicking a button, hovering over an element, or submitting a form. We can listen for these events using the `addEventListener()` method, or by using event attributes like `onclick`, `onmouseover`, `onchange`, etc. The `addEventListener()` method is the preferred way to add event listeners in modern JavaScript.

Here is an example of adding an event listener to a button (using the arrow function syntax):

```
const button = document.querySelector('button')

button.addEventListener('click', () => {
```

```
    alert('Button clicked!')
  })
```

There are many other events you can listen for, like `mouseover`, `mouseout`, `change`, `submit`, etc. You can find a full list of events on the [MDN Web Docs](#).

Each event comes with an `event` object, which holds details such as the target element, the type of event, and other data specific to that interaction. The exact properties vary depending on the event, so a good way to learn what's available is to `console.log` the event object during testing. For deeper reference, the MDN documentation provides a full breakdown of event object properties.

```
button.addEventListener('click', (event) => {
  console.log(event)
  console.log(event.type)
  console.log(event.target)
  console.log(event.x, event.y)
})
```

Before we get into dynamics, these next few things will make life much easier...

Template Literals

Template literals are a newer way to create strings in JavaScript, using backticks (```) instead of quotes (`'` or `"`). This eliminates the need for string concatenation with `+`, making code more readable and less error-prone.

```
const name = "Mike"
const age = 45

// before
const message1 = "Hello, " + name + ". Next year, you'll be " + (age + 1) + "."

// after
const message2 = `Hello, ${name}. Next year, you'll be ${age + 1}.`
```

You can also use template literals to create multi-line strings without the need for `\n`. This is especially useful for creating HTML templates.

```
const poem = `Roses are red  
Violets are blue  
Template literals  
Make strings easier for you`  
  
console.log(poem)
```

Array Methods

These methods encourage functional programming patterns. This can make your code cleaner, easier to maintain, and closer to how developers naturally think about data processing. Some you'll use every day:

- `map()` transforms each item and returns a new array
- `filter()` keeps only items that match a condition
- `reduce()` boils the array down into a single value
- `forEach()` runs a function on each item (no return value)
- `find()` returns the first item that matches a condition
- `some()` and `every()` check if *any* or *all* items match respectively

```
const animals = ["dog", "cat", "elephant", "tiger", "ant"]  
  
animals.map(a => a.toUpperCase()) // ["DOG", "CAT", "ELEPHANT", "TIGER", "ANT"]  
animals.filter(a => a.length > 3) // ["elephant", "tiger"]  
animals.reduce((total, a) => total + a.length, 0) // 22 (length of all animal names)  
animals.forEach(a => console.log(`I love ${a}s!`)) // logs "I love dogs!", "I love  
animals.find(a => a.startsWith("t")) // "tiger"  
animals.some(a => a === "cat") // true  
animals.every(a => a.length >= 3) // false
```

Reminder! When we grab elements using `document.querySelectorAll()`, we'll get a *NodeList*. This needs to be converted into an *Array* before using any of the methods above. You can use `Array.from(items)` or the spread syntax `[...items]` which we'll explain more in depth later.

```
const items = document.querySelectorAll('.item')  
  
Array.from(items).forEach(item => {  
  item.addEventListener('click', () => {  
    console.log('Item clicked:', item.textContent)  
  })  
})
```

```
  })  
})
```

Optional Chaining

Trying to access a property on `null` or `undefined` will throw an error and stop your code. This often happens when you're not sure if some data exists yet, especially when working with APIs or user input.

The optional chaining operator (`?.`) lets you safely "walk" down an object's properties. If the value before `?.` is `null` or `undefined`, it will immediately return `undefined` instead of throwing an error.

```
const user = { name: "John" }  
  
// without optional chaining  
console.log(user.age) // ❌ TypeError  
  
// with optional chaining  
console.log(user?.age) // ✅ undefined
```

This is especially useful for deeply nested objects.

```
const person = {  
  profile: {  
    email: {  
      value: "jane@example.com",  
      verified: true  
    }  
  }  
}  
  
// before  
if (person && person.profile && person.profile.email && person.profile.email.verified) {  
  sendEmail()  
}  
  
// after  
if (person.profile?.email?.verified) {  
  sendEmail()  
}
```

Ok, now onto the fun parts!

Single Delay

Use `setTimeout` to delay the execution of a function. The first argument is the function to execute, and the second argument is the delay in milliseconds.

Remember that functions are first-class citizens in Javascript, so you can pass a function as an argument to another function. Do not invoke the function (i.e. do not use parentheses) when passing it as an argument.

```
setTimeout(() => {  
  console.log('Hello, world!')  
}, 2000)
```

Or, you can pass a named function:

```
const sayHello = () => {  
  console.log('Hello, world!')  
}  
  
setTimeout(sayHello, 2000)
```

Repeated Delays

You can use nested `setTimeout` calls to create multiple delays. Or, you can use `setInterval` to repeat a function at a specified interval. Just like before, the first argument is the function to execute, and the second argument is the delay in milliseconds.

```
setInterval(() => {  
  console.log('Hello, world!')  
}, 2000)
```

Canceling a Delayed Function

Both `setTimeout` and `setInterval` return a unique identifier that you can use to cancel the delayed function. You can pass that same identifier to `clearTimeout` or `clearInterval` respectively to cancel it.

Timeout example:

```
const timeoutId = setTimeout(sayHello, 2000)

clearTimeout(timeoutId)
```

Interval example:

```
const intervalId = setInterval(sayHello, 2000)

clearInterval(intervalId)
```

Task Queue and Event Loop

When you schedule a callback with `setTimeout` or `setInterval`, the function is placed in the **task queue**. The JavaScript runtime only pulls tasks from that queue when the **call stack** is empty. If other synchronous code is still running, your timer's callback must wait, so the delay you request is the *minimum* wait, not a guarantee.

```
console.log('A')

setTimeout(() => {
  console.log('B')
}, 0)

console.log('C')
```

Run the snippet above and you'll always see the synchronous `A` and `C` messages before the timed `B`, even though the delay is set to 0.

Random Selection

Use `Math.random()` to generate a random number between 0 and 1.

```
const random = Math.random()
```

```
console.log(random)
```

To generate a random number between 0 and the length of an array, multiply the random number by the length of the array and round down using `Math.floor`.

```
const items = ['apple', 'banana', 'cherry']
const randomIndex = Math.floor(Math.random() * items.length)
console.log(items[randomIndex])
```

Unfortunately, Javascript doesn't have a nicer way to generate random numbers within a range, but you can always write helper functions to make your code more readable:

```
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

function sample(array) {
  return array[randomInt(0, array.length - 1)]
}

console.log(sample(items))
```

Or use a library like [Lodash](#) that provides additional utility functions.